

3474. Lexicographically Smallest Generated String

Solved 

Hard

 Topics

 Companies

 Hint

You are given two strings, `str1` and `str2`, of lengths `n` and `m`, respectively.

A string `word` of length `n + m - 1` is defined to be **generated** by `str1` and `str2` if it satisfies the following conditions for **each** index $0 \leq i \leq n - 1$:

- If `str1[i] == 'T'`, the **substring** of `word` with size `m` starting at index `i` is **equal** to `str2`, i.e., `word[i..(i + m - 1)] == str2`.
- If `str1[i] == 'F'`, the **substring** of `word` with size `m` starting at index `i` is **not equal** to `str2`, i.e., `word[i..(i + m - 1)] != str2`.

Return the **lexicographically smallest** possible string that can be **generated** by `str1` and `str2`. If no string can be generated, return an empty string `""`.

Example 1:

Input: `str1 = "TFTF"`, `str2 = "ab"`

Output: `"ababa"`

Explanation:

The table below represents the string `"ababa"`

Index	T/F	Substring of length <code>m</code>
0	<code>'T'</code>	<code>"ab"</code>
1	<code>'F'</code>	<code>"ba"</code>
2	<code>'T'</code>	<code>"ab"</code>
3	<code>'F'</code>	<code>"ba"</code>

The strings `"ababa"` and `"ababb"` can be generated by `str1` and `str2`.

Return `"ababa"` since it is the lexicographically smaller string.

Example 2:

Input: `str1 = "TFTF", str2 = "abc"`

Output: `""`

Explanation:

No string that satisfies the conditions can be generated.

Example 3:

Input: `str1 = "F", str2 = "d"`

Output: `"a"`

Constraints:

- `1 <= n == str1.length <= 104`
- `1 <= m == str2.length <= 500`
- `str1` consists only of `'T'` or `'F'`.
- `str2` consists only of lowercase English characters.

Python:

class Solution:

def generateString(self, s: str, t: str) -> str:

n, m = len(s), len(t)

ans = ['?'] * (n + m - 1) # ? indicates a pending position

Process 'T'

for i, b in enumerate(s):

if b != 'T':

continue

The substring must match t

```

    for j, c in enumerate(t):
        v = ans[i + j]
        if v != '?' and v != c:
            return ""
        ans[i + j] = c

old_ans = ans
ans = ['a' if c == '?' else c for c in ans] # Initial default is 'a'

# Process 'F'
for i, b in enumerate(s):
    if b != 'F':
        continue
    # Substring must not equal t
    if "".join(ans[i: i + m]) != t:
        continue
    # Locate the last pending position to modify
    for j in range(i + m - 1, i - 1, -1):
        if old_ans[j] == '?': # Change 'a' to 'b'
            ans[j] = 'b'
            break
    else:
        return ""
return "".join(ans)

```

JavaScript:

```

/***** 🤖 *****/

```

```

const generateString = (pattern, target) => {
    const patternLength = pattern.length;
    const targetLength = target.length;
    const resultLength = patternLength + targetLength - 1;

    const result = Array(resultLength).fill("");
    const locked = Array(resultLength).fill(false);

    // Check if substring matches target
    const matchesTarget = (startIndex) => {
        for (let i = 0; i < targetLength; i++) {
            if (result[startIndex + i] !== target[i]) {
                return false;
            }
        }
    }
    return true;
}

```

```
};
```

```
// Replace last unlocked position with 'b'
const breakMatch = (startIndex) => {
  for (let i = targetLength - 1; i >= 0; i--) {
    const index = startIndex + i;
    if (!locked[index]) {
      result[index] = "b";
      return true;
    }
  }
  return false;
};
```

```
// Step 1: Apply "T" constraints
const applyTrueConstraints = () => {
  for (let i = 0; i < patternLength; i++) {
    if (pattern[i] === "T") {
      for (let j = 0; j < targetLength; j++) {
        const index = i + j;
        const char = target[j];

        if (result[index] === "") {
          result[index] = char;
          locked[index] = true;
        } else if (result[index] !== char) {
          return false;
        }
      }
    }
  }
  return true;
};
```

```
// Step 2: Fill remaining with 'a'
const fillDefaults = () => {
  for (let i = 0; i < resultLength; i++) {
    if (result[i] === "") {
      result[i] = "a";
    }
  }
};
```

```
// Step 3: Apply "F" constraints
```

```

const applyFalseConstraints = () => {
  for (let i = 0; i < patternLength; i++) {
    if (pattern[i] === "F") {
      if (matchesTarget(i) && !breakMatch(i)) {
        return false;
      }
    }
  }
  return true;
};

// Execution flow
if (!applyTrueConstraints()) return "";

fillDefaults();

if (!applyFalseConstraints()) return "";

return result.join("");
};

```

Java:

```

class Solution {
  public String generateString(String S, String t) {
    char[] s = S.toCharArray();
    int n = s.length;
    int m = t.length();
    char[] ans = new char[n + m - 1];
    Arrays.fill(ans, '?'); // '?' indicates undecided positions

    // Process T
    for (int i = 0; i < n; i++) {
      if (s[i] != 'T') {
        continue;
      }
      // Substring must equal t
      for (int j = 0; j < m; j++) {
        char v = ans[i + j];
        if (v != '?' && v != t.charAt(j)) {
          return "";
        }
        ans[i + j] = t.charAt(j);
      }
    }
  }
}

```

```

char[] oldAns = ans.clone();
for (int i = 0; i < ans.length; i++) {
    if (ans[i] == '?') {
        ans[i] = 'a'; // initial value for undecided positions is 'a'
    }
}

// Process F
for (int i = 0; i < n; i++) {
    if (s[i] != 'F') {
        continue;
    }
    // Substring must not equal t
    if (!new String(ans, i, m).equals(t)) {
        continue;
    }
    // Find the last undecided position
    boolean ok = false;
    for (int j = i + m - 1; j >= i; j--) {
        if (oldAns[j] == '?') { // previously filled with 'a', now change to 'b'
            ans[j] = 'b';
            ok = true;
            break;
        }
    }
    if (!ok) {
        return "";
    }
}

return new String(ans);
}
}

```